5-2 Assignment: Binary Search Tree

Malgorzata Debska

Southern New Hampshire University

CS-300 Analysis and Design

David Ostrowski

The provided code aims to implement a binary search tree (BST) in C++ to manage a collection of bids loaded from a CSV file. The code allows users to perform operations such as inserting bids into the tree, removing bids, searching for bids by their unique identifiers, and displaying all bids stored in the tree. Overall, developing the code provided a valuable learning experience in data structure implementation and algorithm design in C++. It required a combination of understanding the problem domain, designing efficient data structures and algorithms, and thorough testing to ensure correctness and reliability.

Pseudocode:

Structure Node:

    Data: bid

    LeftChild: Node*

    RightChild: Node*

Structure BinarySearchTree:

    Root: Node*

Function Insert(bid):

    Create new node with bid data

    If tree is empty:

```
    Set new node as root
Else:
    CurrentNode = Root
    Loop until insertion point found:
        If bid's key < CurrentNode's key:
            If CurrentNode has no left child:
                Set new node as left child of CurrentNode
                Break loop
            Else:
                Move to left child
        Else if bid's key > CurrentNode's key:
            If CurrentNode has no right child:
                Set new node as right child of CurrentNode
                Break loop
            Else:
                Move to right child
        Else:
            Update bid data of CurrentNode with new bid data
```

```
            Break loop


Function Remove(bidId):
    If tree is empty:
        Return
    Else:
        Call recursive function RemoveHelper(Root, bidId)


Function RemoveHelper(CurrentNode, bidId):
    If CurrentNode is null:
        Return null
    If bidId < CurrentNode's bidId:
        Set left child to RemoveHelper(CurrentNode's left child, bidId)
    Else if bidId > CurrentNode's bidId:
        Set right child to RemoveHelper(CurrentNode's right child, bidId)
    Else:
        If CurrentNode has no left child:
            Return right child
```

```
        Else if CurrentNode has no right child:
            Return left child
        Else:
            Find minimum value node in right subtree
            Copy its data to CurrentNode
            Set right child to RemoveHelper(CurrentNode's right child, copiedNode's bidId)
    Return CurrentNode

Function Search(bidId):
    Call recursive function SearchHelper(Root, bidId)

Function SearchHelper(CurrentNode, bidId):
    If CurrentNode is null or CurrentNode's bidId is bidId:
        Return CurrentNode
    If bidId < CurrentNode's bidId:
        Return SearchHelper(CurrentNode's left child, bidId)
    Else:
        Return SearchHelper(CurrentNode's right child, bidId)
```

```
Function DisplayAllBids():
    Call recursive function DisplayHelper(Root)

Function DisplayHelper(CurrentNode):
    If CurrentNode is not null:
        Display CurrentNode's bid data
        Recursively call DisplayHelper(CurrentNode's left child)
        Recursively call DisplayHelper(CurrentNode's right child)
```